P2 - Reliable Data Transfer

Missing 100 Points Possible

10/9/2026

Attempt 1 **NEXT UP: Review Feedback**

Offline Score: N/A Add Comment

Unlimited Attempts Allowed

10/11/2026

Details

> **Week 7 · 100 points · about 15 hours · submit in Canvas**

Overview

You are going to build a reliable file transfer on top of UDP, through a relay that deliberately loses, corrupts, and duplicates what you hand it. This is the rdt2.2 / rdt3.0 / Go-Back-N progression from chapter 3, implemented rather than drawn on a whiteboard. By the end you will understand why TCP is shaped the way it is.

Fork the **starter repository (https://github.com/shanep/makefile-project-starter)** using the **Use this template** button and name your copy `cs425-p2`.

> **Warning**
>
> Your code must compile on **GitHub Codespaces and Onyx**. If it compiles on only one of them you will receive a zero even if it works on the other.

Learning Outcomes

- 3.1 Implement a reliable data transfer protocol over an unreliable channel
- 3.3 Choose between TCP and UDP for a given application and defend the choice
- 3.4 Analyze a complex computing problem and apply principles of computing to identify solutions **(ABET Outcome 1)**

Background

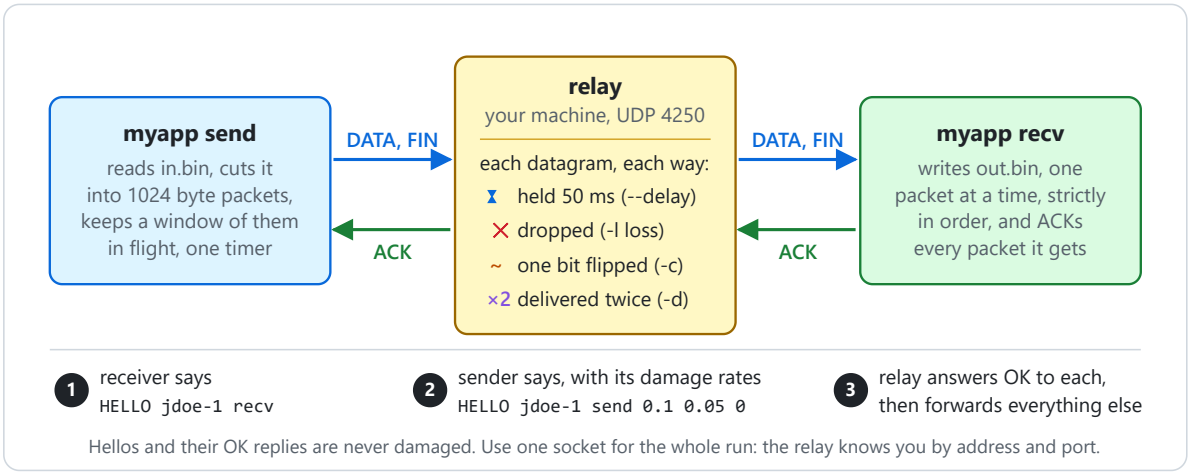
UDP gives you datagrams and nothing else. A datagram can vanish, arrive with bits flipped, or arrive twice, and the socket API will not tell you which happened. Every guarantee TCP makes has to be built on top by the two endpoints. In this project you build the core of it:

- A **checksum** so the receiver can tell a damaged packet from a good one.
- **Sequence numbers** so the receiver can tell a new packet from a retransmission.
- **Cumulative acknowledgments** so the sender knows what has arrived.
- A **timer** so the sender notices when something was lost, since a lost packet sends no error message.
- A **window** so the sender can have several packets in flight at once, instead of sitting idle for a full round trip after every packet.

The protocol is **Go-Back-N**, as described in section 3.4 of the textbook and in the chapter 3 notes. Reread both before you start. You will need the sender and receiver state machines in front of you, not from memory.

The setup

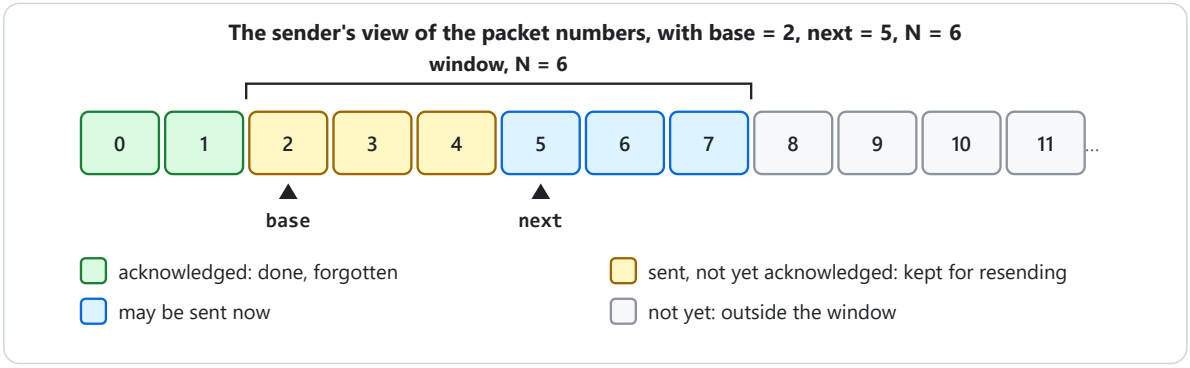
You write one program with two modes. The **receiver** registers with the relay and writes what it receives to a file. The **sender** registers with the relay and sends a file. The relay pairs them up and forwards traffic in both directions, damaging it along the way at the rates the sender asked for.



DATA packets carry the file, a **FIN** marks its end, and an **ACK** tells the sender what has arrived. **The packet format** below has the details.

The sender's window

The whole sender is two numbers and the packets between them. `base` is the oldest packet not yet acknowledged, and `next` is the next packet never sent. Everything below `base` is finished. Everything from `base` up to `next - 1` is in flight, so the sender keeps a copy in case it has to resend it. The sender may keep sending until `next` reaches `base + N`, and then it has to wait for an ACK to move `base`.



Every rule in Task 3 is a statement about how `base` and `next` move. An ACK moves `base` to the right, which opens room for `next` to follow. A timeout moves nothing: it resends everything from `base` to `next - 1`.

Go-Back-N recovering from a loss

This is one complete exchange with a window of 4 and six DATA packets, where the relay drops DATA 2. Read it top to bottom, one arrow at a time.



Three things in it are the whole protocol:

- **The receiver never skips ahead.** DATA 3, 4 and 5 arrive intact, but the receiver is waiting for 2, so it throws them away and repeats ACK 2 each time. That keeps the receiver trivial: it holds one number and writes bytes in order.
- **Duplicate ACKs change nothing.** ACK 2 does not move `base` past 2, so the sender ignores each repeat. It cannot tell a lost packet from a slow one, so it waits for its timer.
- **A timeout goes back to `base`.** When the timer runs out, the sender resends 2, 3, 4 and 5, even though the receiver had already seen 3, 4 and 5 once. That wasted work is the price of the simple receiver, and Task 6 has you measure it.

The relay

The relay is a small program you run yourself, on the same machine as your sender and receiver. Download **cs425_relay.py (https://boisestatecanvas.instructure.com/courses/48194/files/25324577)** . It is a single Python file that needs nothing but `python3`, which Onyx, Codespaces, macOS and Linux all have. Run it in one terminal, then your receiver and your sender in two more, all pointed at `127.0.0.1`:

```
python3 cs425_relay.py --delay 50
./build/release/myapp recv -s me 127.0.0.1 out.bin
./build/release/myapp send -s me -w 16 -l 0.1 -c 0.05 127.0.0.1 in.bin
```

Use `127.0.0.1` wherever this page says `<relay>`. The relay listens on UDP port **4250** and only accepts traffic from the machine it runs on.

- `--delay 50` holds every datagram for 50 ms, so the round trip is 100 ms, about what a trip across the western United States takes. Without it, a loopback round trip is a few microseconds and your window has nothing to hide. Task 6 requires it.

- `--port`: Onyx is shared, and only one program can own a port. If the relay says the port is already in use, someone else got there first. Pick a port of your own between 20000 and 60000, start the relay with `--port <n>`, and pass the same number to your sender and receiver with `-p <n>`.
- `--seed 7` makes the damage the same every run, so when a transfer fails you can replay exactly that transfer while you debug it.
- `-v` logs more. Either way, when a session ends the relay prints how many datagrams went each way and how many it dropped, corrupted and duplicated, which tells you whether your packets reached it at all.

Press `Ctrl-C` to stop it. Your unit tests do not need the relay at all, so `make check` and CI work anywhere.

Registering

Before any data moves, each side sends the relay one plain-text datagram, with no newline, and waits for the reply:

```
receiver:  HELLO <session> recv
sender:    HELLO <session> send <loss> <corrupt> <dup>
```

- `<session>` is a name you choose, 1 to 32 characters from `a-z`, `0-9` and `-`. Use your BroncoName with a suffix, for example `jdoe-1`. Both sides must use the same name.
- `<loss>`, `<corrupt>` and `<dup>` are probabilities between `0` and `0.5`, written as decimals, for example `HELLO jdoe-1 send 0.1 0.05 0`.
- The relay answers `OK`, or `ERR <reason>` when something is wrong, such as a malformed hello or a sender that registers before its receiver. Print the reason and exit.
- Starting a new receiver replaces the session, so a run you killed halfway does not lock you out of your session name.

The relay never damages a `HELLO` or its reply, but UDP makes no promises, even between two programs on one machine. If no reply arrives within one second, send the hello again, up to five attempts in all.

Register the receiver first. The sender's hello fails with `ERR no receiver` if the receiver is not there yet.

Forwarding

Once both sides hold an `OK`, every datagram the relay gets from one member of the session goes to the other, unchanged except for what the rates ask for. Each datagram, in each direction, is handled independently:

1. With probability `loss`, it is dropped.
2. Otherwise, with probability `corrupt`, one randomly chosen bit is flipped.
3. Otherwise, with probability `dup`, it is delivered twice.

ACKs are damaged the same way as data, so your sender will see lost ACKs, and your receiver will see duplicate packets, and both have to cope.

The relay knows the two sides by the address and port they sent their hello from, so each side must use **one socket for the whole run**. Do not open a new socket per packet. A session is forgotten after 30 seconds with no traffic.

> **Tip**
>
> If your hello gets no reply at all, check that the relay is running and that your `-p` matches its `--port`. A relay that is not there presents as silence, not as an error.

The packet format

Every datagram you send through the relay after registering is one packet, laid out exactly as shown here. Multi-byte fields are in **network byte order**, so use `htons`, `htonl` and their inverses. Do not `memcpy` a struct onto the wire. Padding and byte order make that non-portable, and portability is graded.

Offset	Size	Field
0	1	`type`: `0` DATA, `1` ACK, `2` FIN
1	1	reserved, always `0`
2	2	`checksum`
4	4	`seq`
8	2	`length` of the payload in bytes
10	`length`	payload, at most **1024** bytes

The header is 10 bytes, so the largest packet is 1034 bytes.

Checksum. Use the Internet checksum from **RFC 1071 (https://www.rfc-editor.org/rfc/rfc1071)** : the 16-bit one's complement of the one's complement sum of the packet taken as 16-bit words, computed with the `checksum` field set to zero. Pad an odd length with one zero byte for the calculation only. The RFC's worked example is a good first unit test: the bytes `00 01 f2 03 f4 f5 f6 f7` sum to `0xddf2`, so their checksum is `0x220d`.

Sequence numbers count packets, not bytes.

- A DATA packet's `seq` is its index in the file, starting at 0. Every DATA packet carries 1024 bytes of the file except the last, which carries what is left.
- The FIN packet's `seq` is the number of DATA packets, which is the next index after the last one. An empty file is a single FIN with `seq` 0.
- An ACK's `seq` is the index of the **next packet the receiver expects**. It says that every packet before `seq` has arrived. ACK 5 acknowledges packets 0 to 4. ACK and FIN packets have `length` 0.

Files in this project are at most 16 MiB, so a 32-bit `seq` never wraps and you do not need modular arithmetic.

Validating. A packet is discarded silently, and treated exactly as if it were lost, unless all of these hold:

- the datagram is at least 10 bytes;
- `10 + length` equals the datagram size, and `length` is at most 1024;
- `type` is 0, 1 or 2, and `reserved` is 0;
- the checksum verifies.

There is no NAK, and a bad packet gets no reply at all. The sender's timer recovers a damaged packet exactly as it recovers a lost one.

> **Tip**
>
> Check `length` against the size `recvfrom` actually returned before you copy a single payload byte. Trusting a length field from the network is how a buffer overflow is born, and the relay will flip bits in that field for you.

Worked examples

Example 1: cutting a file into packets

A 2500-byte file becomes three DATA packets and a FIN:

Packet	`seq`	`length`	Carries
DATA	0	1024	bytes 0 to 1023
DATA	1	1024	bytes 1024 to 2047
DATA	2	452	bytes 2048 to 2499
FIN	3	0	nothing: "that was all of it"

On a clean channel the receiver answers with ACK 1, ACK 2, ACK 3, and then ACK 4 for the FIN. The sender exits once ACK 4 arrives, and the receiver exits after its two-second linger.

Example 2: one packet, byte by byte

DATA packet 2 carrying the three bytes `Hi!` is 13 bytes on the wire:

```
00           type      DATA
00           reserved
96 91        checksum
00 00 00 02  seq       2
00 03        length    3
48 69 21     payload   "Hi!"
```

To compute the checksum, set the checksum field to zero, pad the odd length with one zero byte, and add the packet up as 16-bit words, folding any carry back in:

```
0000 + 0000 + 0000 + 0002 + 0003 + 4869 + 2100 = 696e
one's complement of 696e = 9691
```

The receiver checks it by adding up the same words with the checksum left in. An undamaged packet always sums to `ffff`: here `696e + 9691 = ffff`. Anything else means a bit changed on the way. For comparison, ACK 3 is ten bytes: `01 00 fe fc 00 00 00 03 00 00`.

Example 3: a lost ACK costs nothing

The sender has sent DATA 2, 3 and 4 (`base` is 2, `next` is 5), and all three arrive. The receiver sends ACK 3, ACK 4 and ACK 5, and the relay drops ACK 3 and ACK 4. When ACK 5 arrives, the sender sets `base` to 5 in one step. An ACK says "everything below this number arrived", so a later ACK covers every earlier one that went missing. Nothing is resent.

Example 4: a corrupted packet is a lost packet

The relay flips one bit in DATA 7. The receiver computes the checksum, gets something other than `ffff`, and discards the packet without replying, exactly as if it never arrived. From the sender's side the two cases look the same, and they get the same fix: the timer runs out and DATA 7 goes again. A flipped bit in an ACK works the same way: the sender discards it, and the next good ACK covers it, as in Example 3.

Example 5: why the receiver lingers

The file has three DATA packets, so the FIN is `seq` 3. The receiver gets it, closes the file and sends ACK 4, and the relay drops that ACK. The sender's timer runs out and it sends FIN 3 again.

- **With the linger**, the receiver is still listening. It answers the repeated FIN with ACK 4 again, the sender exits 0, and so does the receiver.
- **Without it**, the receiver has already exited. Nobody answers the repeated FINs, and after 10 timeouts in a row the sender gives up and exits 2, even though the whole file arrived.

Task 1 - The command line

Your program is built as `./build/release/myapp` and must take this command line:

```
Usage: myapp send -s <session> [-w window] [-T timeout-ms] [-l loss]
                  [-c corrupt] [-d dup] [-p port] <relay> <file>
       myapp recv -s <session> [-p port] <relay> <file>

  -s <session>     session name shared by the sender and the receiver
  -w <window>      Go-Back-N window size in packets, 1 to 64 (default: 8)
  -T <timeout-ms>  retransmission timeout in milliseconds (default: 250)
  -l <loss>        probability the relay drops a packet (default: 0)
  -c <corrupt>     probability the relay flips a bit (default: 0)
  -d <dup>         probability the relay duplicates a packet (default: 0)
  -p <port>        relay port (default: 4250)
  <relay>          host name or address of the relay
  <file>           file to send, or file to write what is received
```

With the relay running, in two more terminals, receiver first:

```
./build/release/myapp recv -s jdoe-1 127.0.0.1 out.bin
./build/release/myapp send -s jdoe-1 -w 16 -l 0.1 -c 0.05 127.0.0.1 in.bin
cmp in.bin out.bin && echo identical
```

Read the mode from `argv[1]`, then parse the rest with `getopt`. The interface is fixed because it is what is exercised when your project is graded, so match it exactly. Resolve `<relay>` with `getaddrinfo`; do not assume it is a dotted quad.

Exit **0** on a successful transfer, **1** when the command line is wrong, and **2** when the relay refuses you, the network fails, or the transfer gives up. When run with no arguments at all, print the usage message and exit **0**. This is what `make leak` runs, so it has to be a clean, successful path.

Task 2 - The receiver

After registering, the receiver keeps one number, `expected`, the index of the next packet it wants, starting at 0. For every packet that passes validation:

- A DATA packet with `seq == expected`: write its payload to the file, add one to `expected`, and send `ACK expected`.
- Any other DATA packet, whether a duplicate or one that arrived ahead of a gap: discard it and send `ACK expected` again. Go-Back-N receivers do not buffer out-of-order packets.
- A FIN with `seq == expected`: close the file, add one to `expected`, and send `ACK expected`. Then **linger** for two seconds, answering any repeated FIN with the same ACK, and exit 0.

The linger exists because the receiver's last ACK can be lost. Without it the sender retransmits its FIN to nobody and eventually gives up on a transfer that actually succeeded. This is the same problem TCP's TIME_WAIT state solves.

If nothing valid arrives for 30 seconds, the receiver gives up and exits 2.

Task 3 - The sender

Stop-and-wait (rdt3.0) is Go-Back-N with a window of 1, so if your sender is right for `-w 1` and for `-w 16`, it is right. The sender keeps:

- `base`, the oldest packet not yet acknowledged;
- `next`, the next packet not yet sent;
- a copy of every packet from `base` to `next - 1`, for retransmission;
- **one** timer, running whenever anything is unacknowledged.

It follows these rules:

- **Send** while `next < base + window` and there is still data: send packet `next` and add one to `next`. Start the timer if it was not running.
- **On an ACK** with `seq > base`: set `base = seq`, which may slide the window by several packets at once. That is what cumulative means. Restart the timer if anything is still unacknowledged, and stop it if not. An ACK with `seq <= base` is a duplicate: ignore it.
- **On a timeout**: resend **every** packet from `base` to `next - 1`, and restart the timer. This is the "go back" in Go-Back-N.
- **When every DATA packet is acknowledged**, send the FIN, and treat it like one more packet: retransmit it on timeout until its ACK arrives, then exit 0.
- **Give up** after 10 timeouts in a row with no progress, which means no ACK that moved `base`. Say so and exit 2.

Wait for ACKs and timeouts together with `poll` on the socket, using the time left on the timer as the `poll` timeout. Measure time with `clock_gettime(CLOCK_MONOTONIC, ...)`, never with `time()` or the wall clock, which can jump.

Task 4 - Design for testability

You cannot unit test against a relay that loses packets at random, so the protocol logic must not touch a socket, a clock, or a file. This separation is most of the design work in this project and it is graded. Split `src/lab.h` into three layers:

1. **Packets.** Compute a checksum, encode a packet into a buffer, and decode and validate a buffer into a packet. Pure functions: bytes in, bytes or a struct out.
2. **The Go-Back-N state machines.** A sender and a receiver, each a struct plus the functions that feed it events: an ACK arrived, a DATA packet arrived, the timer expired. The current time is always **passed in** as a parameter in milliseconds, and each function returns what to do next: which packets to send, what payload to deliver, and when the timer is due. Nothing in here calls `sendto`, `recvfrom`, `poll`, `clock_gettime`, or `fwrite`.
3. **The I/O.** The socket, the relay hello, the `poll` loop, the clock, and the file. This is the only layer that knows any of them exist, and it should be thin: read an event, hand it to the state machine, carry out what comes back.

The payoff is layer 2. Your real program drives it with a socket and a clock, and your tests drive it with a fake clock and an in-memory channel you control completely. A lost packet in a test is just a packet you did not pass along, and a timeout is just a larger number for "now".

Task 5 - Testing

Add Unity tests for every function you declare in `src/lab.h`.

```
make check
```

Beyond the happy path, make sure you have tests for:

- the RFC 1071 example above, a packet with an odd length, and a single flipped bit that the checksum catches;
- a datagram that is too short, one whose `length` does not match its size, and one with an unknown `type`;
- a receiver given a duplicate, a packet from beyond a gap, and a repeated FIN;
- a sender whose window is full, a cumulative ACK that slides it several packets at once, a duplicate ACK, a timeout that resends the whole window, and one that gives up after 10 timeouts;
- an empty file, and a file that is an exact multiple of 1024 bytes;
- a **complete transfer** between your sender and receiver state machines through an in-memory channel that drops, corrupts and duplicates 20% of the packets in each direction, using a random number generator with a fixed seed, with a check that the bytes delivered match the bytes sent.

That last test is the one that tells you your protocol works. If it passes with several seeds, the relay holds no surprises.

Task 6 - Measure the window

Make a 1 MiB test file:

```
head -c 1048576 /dev/urandom > 1mib.bin
```

Run these through the relay started with `--delay 50`, as shown in **The relay**, so that everyone measures against the same 100 ms round trip. Transfer the file with the default 250 ms timeout, three times for each combination below, checking every copy with `cmp`, and time each run with `time`:

Window	Loss	Corrupt	Dup
1	0	0	0
16	0	0	0
1	0.05	0	0
16	0.05	0	0

Add a **Results** section to your `README.md` with a table of the mean time and the throughput in KiB/s for each combination. Then answer in a paragraph or two:

- From the window 1, no loss run, compute the round trip time your sender actually saw. (It sent 1025 packets and waited one round trip for each.) The relay adds 100 ms. Where does the rest come from?
- Use that round trip time to explain the speedup at window 16. Is it close to 16 times? Why or why not?
- Compared with its no loss time, why does 5% loss slow the window 16 run down so much more than the window 1 run? Think about what a timeout makes the sender resend in each case.

Task 7 - Coverage

```
make clean
make all
make report
```

Fix your tests until you have 100% coverage with everything passing. As in P0, you may only exclude branches originating from system or library calls.

Task 8 - Leak and crash check

- Run `make leak`, then `make leak-test`.
- Fix every leak and every crash. Retransmission buffers and the error paths are exactly where the allocations get missed.

Task 9 - Replace README.md

Replace `README.md` following this **example (https://gist.github.com/shanep/4fc7962a3ac80349094d50e0fa57cf6e)** . Your Design section should explain the three-layer split from Task 4 and why it is there, and the Results section from Task 6 goes after it.

Task 10 - Continuous integration

- Push everything to GitHub and confirm the CI run is green.
- Run the **Create Submission Report Via GitHub Action** workflow.
- Download `submission-report.docx` once it completes.

Submitting

1. Download `submission-report.docx` from GitHub and submit it to Canvas.
2. Check your submission in Canvas so you know it arrived intact.

Rubric

#	Criterion	Points
1	Packets encoded to the specified format; Internet checksum correct; damaged and malformed datagrams rejected	15
2	Required command line; relay registration with retries; a file transfers byte-identical over a clean channel	10
3	Receiver delivers in order, sends cumulative ACKs, and discards and re-ACKs duplicates and out-of-order packets	15
4	Sender keeps a window with one timer, slides on cumulative ACKs, and resends the whole window on timeout, recovering from lost, corrupted and duplicated packets and ACKs	20
5	Transfer ends cleanly: FIN acknowledged, receiver lingers, sender gives up with exit 2 after 10 fruitless timeouts	5
6	Results table for the four runs, with the round trip estimate and the three questions answered	10
7	Protocol logic separated from socket, clock and file I/O; Unity tests, including a seeded lossy end-to-end transfer; 100% coverage	15

#	Criterion	Points
8	No memory leaks or crashes under `make leak-test`; `README.md` replaced; CI green on the last push	10

View Rubric

P2 - Reliable Data Transfer rubric (1)

Criteria	Ratings		Points
Packets encoded to the specified format; Internet checksum correct; damaged and malformed datagrams rejected	Full marks 15 pts	No marks 0 pts	/15 pts
Required command line; relay registration with retries; a file transfers byte-identical over a clean channel	Full marks 10 pts	No marks 0 pts	/10 pts
Receiver delivers in order, sends cumulative ACKs, and discards and re-ACKs duplicates and out-of-order packets	Full marks 15 pts	No marks 0 pts	/15 pts
Sender keeps a window with one timer, slides on cumulative ACKs, and resends the whole window on timeout, recovering from lost, corrupted and duplicated packets and ACKs	Full marks 20 pts	No marks 0 pts	/20 pts
Transfer ends cleanly: FIN acknowledged, receiver lingers, sender gives up with exit 2 after 10 fruitless timeouts	Full marks 5 pts	No marks 0 pts	/5 pts
Results table for the four runs, with the round trip estimate and the three questions answered	Full marks 10 pts	No marks 0 pts	/10 pts
Protocol logic separated from socket, clock and file I/O; Unity tests, including a seeded lossy end-to-end transfer; 100% coverage	Full marks 15 pts	No marks 0 pts	/15 pts
No memory leaks or crashes under `make leak-test`; `README.md` replaced; CI green on the last push	Full marks 10 pts	No marks 0 pts	/10 pts